

Informe de entrega

Taller 1: Prompts Engineering

Crear una aplicación de Lista de Tareas en Python desde la terminal utilizando prompt engineering.

Estudiante: Jhossua Vega

Github:

https://github.com/avegac1996/Taller1_prompts_engineering.git

Julio de 2026

Introducción

Este informe presenta el desarrollo de una aplicación profesional de Lista de Tareas en Python, ejecutable desde la terminal. El objetivo fue aplicar técnicas de prompt engineering para generar un entregable robusto, ordenado y con enfoque de calidad (QA) y experiencia de usuario (UX).

El trabajo se organizó en cuatro prompts estructurados que cubren: arquitectura y pruebas, interfaz de línea de comandos, funcionalidades avanzadas y filtros, y finalmente UX, colores y documentación. Cada prompt incluye secciones de rol, contexto, tarea, restricciones, criterios de aceptación QA y consideraciones UX.

El resultado final es un programa modular, con una clase TaskManager que gestiona la persistencia JSON, y una interfaz CLI basada en argparse. El proyecto incluye pruebas unitarias, validaciones, manejo de errores, exportación CSV y está alojado en GitHub para su revisión.

2. Proveedor y modelo de IA

- **Proveedor:** Cascade (asistente de código integrado en el IDE)
- **Modelo de IA:** OpenAI GPT-4o

3. Repositorio en GitHub

URL: https://github.com/avegac1996/Taller1_prompts_engineering.git

4. Prompts utilizados

Se diseñaron cuatro prompts estructurados con enfoque de QA y UX. El detalle completo se presenta a continuación.

Prompts utilizados para construir la aplicación

A continuación, se presentan los prompts estructurados que se usaron para generar la aplicación profesional de Lista de Tareas. Cada prompt sigue la guía de **El buen prompt** ampliada con secciones de **QA** y **UX** para garantizar un entregable robusto, ordenado y fácil de usar.

Prompt 1 - Arquitectura, dominio, persistencia y pruebas

```
[1. Rol]
Actúa como Ingeniero de Software Senior especializado en Python, aplicaciones CLI y desarrollo guiado por pruebas (TDD).

[2. Contexto]
Estoy desarrollando una aplicación de Lista de Tareas (Todo List) que se ejecuta exclusivamente desde la terminal, para una entrega universitaria de Prompt Engineering. El entorno objetivo es Python 3.14 en Windows. El proyecto debe ser mantenible, testable y listo para subir a GitHub.

[3. Tarea]
Diseña la capa de dominio y persistencia de la aplicación en un archivo llamado todo.py:
- Crea una clase TaskManager que gestione las tareas en un archivo JSON ubicado en el directorio personal del usuario.
- Implementa los métodos: add, get, complete, delete, edit, list, search, stats y export_csv
- Define validaciones para prioridad (baja, media, alta) y fecha límite (AAAA-MM-DD).
- Maneja JSON corrupto sin detener la aplicación.
- Crea la carpeta tests/ con pruebas unitarias usando unittest y archivos temporales, cubriendo: CRUD, validaciones, persistencia y búsqueda.

[4. Restricciones]
- Usa únicamente la biblioteca estándar de Python en la lógica de negocio.
- Separa responsabilidades: la clase TaskManager no debe depender de argparse ni de la interfaz de usuario.
- Los IDs deben ser únicos, secuenciales y persistentes.
- El código debe estar comentado y seguir buenas prácticas de nombres y estructura.

[5. Criterios de aceptación QA]
- Las 23+ pruebas unitarias deben ejecutarse con `python -m unittest discover tests` y pasar todas.
- Si el archivo JSON está corrupto, la aplicación debe recuperarse devolviendo una lista vacía.
- La validación de prioridad y fecha debe lanzar ValueError con mensajes claros.
```

```
[6. Consideraciones UX]
- Los mensajes de error deben dirigirse a stderr y usar códigos de salida distintos de cero.
- Las operaciones exitosas deben mostrar confirmaciones concisas.

[7. Formato]
Devuélveme el archivo todo.py completo y comentado, junto con el contenido de tests/test_todo.py, en bloques de código separados y sin explicaciones adicionales.
```

Resultado esperado

- Clase TaskManager con persistencia JSON en %USERPROFILE%\todo_cli\tasks.json.
 - Métodos de negocio: add, get, complete, delete, edit, list, search, stats, export_csv.
 - Validaciones de prioridad y fecha.
 - Suite de pruebas tests/test_todo.py con cobertura de CRUD, validaciones y persistencia.
-

Prompt 2 - Interfaz de línea de comandos (CLI)

```
[1. Rol]
Actúa como Ingeniero de Software Senior especializado en Python y CLI tools.

[2. Contexto]
Ya tengo la clase TaskManager del Prompt 1 con persistencia JSON, validaciones y pruebas unitarias. Ahora necesito exponer la funcionalidad al usuario mediante una interfaz de terminal profesional.

[3. Tarea]
Construye la interfaz CLI de la aplicación en todo.py:
- Usa argparse con subcomandos: add, list, done, delete, edit, search, stats y export.
- Cada subcomando debe invocar los métodos de TaskManager y mostrar resultados claros.
- Maneja errores de validación (prioridad o fecha inválida) y errores de negocio (ID no encontrado) con mensajes en stderr y salida 1.
- Asegura que el programa se pueda ejecutar con `python todo.py --help` y que la ayuda sea legible.

[4. Restricciones]
- Mantén la clase TaskManager separada de la CLI; no mezcles lógica de argumentos con lógica de negocio.
- No agregues dependencias externas para la CLI.
- Usa nombres de argumentos consistentes y descripciones en español.

[5. Criterios de aceptación QA]
- `python todo.py --help` muestra todos los subcomandos y sus opciones.
- `python todo.py done 999` devuelve un mensaje de error y código de salida 1.
- `python todo.py add "Tarea" --priority invalida` devuelve error de validación.
- La ejecución manual de los comandos principales debe ser exitosa.

[6. Consideraciones UX]
- La ayuda debe estar en español y ser concisa.
- Las operaciones exitosas deben confirmar la acción con el ID y título de la tarea.
- Los errores deben indicar exactamente qué salió mal y cómo corregirlo.

[7. Formato]
Devuélveme el archivo todo.py completo y actualizado, con la CLI integrada y comentada, sin explicaciones adicionales.
```

Resultado esperado

- todo.py con clase TaskManager + funciones cmd_* y main() basada en argparse.
- Subcomandos: add, list, done, delete, edit, search, stats, export.
- Manejo de errores con stderr y códigos de salida.

Prompt 3 - Funcionalidades avanzadas, filtros y control de calidad

```
[1. Rol]
Actúa como Ingeniero de Software Senior y QA Lead en Python CLI.

[2. Contexto]
La aplicación ya tiene capa de dominio y CLI básica. Necesito agregar funcionalidades avanzadas que permitan al usuario gestionar y consultar tareas de forma flexible, sin perder robustez ni testabilidad.

[3. Tarea]
Extiende todo.py y tests/test_todo.py con:
- Campos prioridad (baja, media, alta) y fecha límite (AAAA-MM-DD) en add y edit.
- Filtros en list: todas, pendientes, completadas, hoy (fecha límite igual a hoy), vencidas (fecha límite pasada y no completada).
- Ordenamiento en list: por fecha de creación, prioridad o fecha límite.
- Búsqueda case-insensitive por palabra clave en título o descripción.
- Detección de tareas vencidas y conteo en stats.
- Tests unitarios adicionales para cada filtro, ordenamiento y detección de vencidas.

[4. Restricciones]
- Usa argparse choices para restringir los valores válidos de prioridad y filtros.
- La fecha límite se almacena como string ISO AAAA-MM-DD.
- La lógica de filtros debe estar en TaskManager, no en la CLI.
- No alteres el formato JSON existente; los nuevos campos son opcionales.

[5. Criterios de aceptación QA]
- `python todo.py list --status vencidas` muestra solo tareas pendientes con fecha límite anterior a hoy.
- `python todo.py list --priority alta` muestra solo tareas de alta prioridad.
- `python todo.py list --sort fecha` ordena por fecha límite.
- `python todo.py stats` muestra el total, completadas, pendientes, vencidas y progreso.
- La suite de tests unitarios cubre al menos 20 casos y todos pasan.

[6. Consideraciones UX]
- La salida de list debe ser una tabla alineada con columnas: estado, ID, título, prioridad, fecha límite.
- Las fechas vencidas deben resaltarse visualmente cuando estén activas (se implementará con color en el siguiente prompt).
- Los filtros vacíos deben mostrar un mensaje claro: "No hay tareas que coincidan con el filtro."

[7. Formato]
Devuélveme el archivo todo.py completo y actualizado, junto con el archivo tests/test_todo.py completo, en bloques de código separados y sin introducciones.
```

Resultado esperado

- Filtros --status con opciones: todas, pendientes, completadas, hoy, vencidas.
- Filtros --priority y --sort.
- Búsqueda case-insensitive.
- Detección de tareas vencidas en stats y list.
- Suite de pruebas ampliada.

Prompt 4 - UX profesional, colores, exportación CSV y documentación

```
[1. Rol]
Actúa como Ingeniero de Software Senior con enfoque en UX/UI de terminal y documentación técnica.
```

```
[2. Contexto]
La aplicación de Lista de Tareas ya tiene dominio, CLI, filtros y pruebas. El entregable final debe ser profesional, visualmente claro y acompañado de documentación completa para subir a GitHub.

[3. Tarea]
Politea la experiencia de usuario y prepara la entrega:
- Agrega soporte de colores opcional usando colorama. Si colorama no está instalado, la aplicación funciona sin colores.
- Usa colores semánticos: alta (rojo), media (amarillo), baja (verde). Las fechas vencidas de tareas pendientes deben mostrarse en rojo.
- Mejora el formato de list para que sea legible, alineado y no use emojis.
- Implementa el comando export para generar un CSV UTF-8 con encabezados.
- Crea el README.md con instalación, uso, ejemplos de comandos y sección de calidad/tests.
- Crea el requirements.txt con colorama como dependencia opcional.
- Crea el .gitignore para Python y archivos CSV generados.
- Crea el informe.md de entrega con: proveedor/modelo de IA, prompts utilizados, resultado final, enlace a GitHub e instrucciones para push.
- Crea el capturas.md con las pruebas manuales en terminal por cada prompt.

[4. Restricciones]
- colorama es la única dependencia opcional y debe estar en requirements.txt.
- El CSV debe ser legible y usar codificación UTF-8.
- La documentación debe ser clara, profesional y sin explicaciones excesivas.
- No se deben exponer secretos ni credenciales en la documentación.

[5. Criterios de aceptación QA]
- `pip install -r requirements.txt` instala colorama sin errores.
- Con colorama instalado, `python todo.py list` muestra prioridades en colores.
- Sin colorama, el programa funciona exactamente igual.
- `python todo.py export tareas.csv` genera un archivo CSV correcto.
- El README e informe contienen toda la información requerida por la rubrica.

[6. Consideraciones UX]
- La salida debe ser amigable en Windows Terminal, CMD y PowerShell.
- Los mensajes de error deben indicar el comando correcto o el formato esperado.
- El README debe incluir ejemplos de comandos copy-pasteables.

[7. Formato]
Devuélveme el contenido completo de los archivos: todo.py, README.md, requirements.txt, .gitignore, informe.md y capturas.md, en bloques de código separados y sin introducciones.
```

Resultado esperado

- todo.py con colores opcionales, formato profesional, export CSV, stats completos.
- README.md, requirements.txt, .gitignore, informe.md, capturas.md.
- Proyecto listo para ejecutar, probar y subir a GitHub.

5. Resultado final

La aplicación entregada es una Lista de Tareas profesional que se ejecuta completamente desde la terminal. Incluye capa de dominio (clase TaskManager), interfaz CLI con argparse, validaciones, filtros, estadísticas, exportación CSV y una suite de pruebas unitarias.

5.1. Código fuente - todo.py

El siguiente listado muestra el archivo principal de la aplicación:

```
"""Lista de Tareas - Aplicación CLI profesional en Python."""

import argparse
import csv
import datetime
import json
import sys
```

```

from pathlib import Path

# Colorama es opcional; si no está instalado la aplicación sigue funcionando.
try:
    from colorama import init, Fore, Style
    init(autoreset=True)
    _HAS_COLOR = True
except Exception: # pragma: no cover
    _HAS_COLOR = False

    class _DummyColor:
        def __getattr__(self, name):
            return ""

    Fore = _DummyColor()
    Style = _DummyColor()

# Ubicación por defecto de la base de datos JSON.
DEFAULT_DATA_DIR = Path.home() / ".todo_cli"
DEFAULT_DATA_FILE = DEFAULT_DATA_DIR / "tasks.json"

ESTADO_PENDIENTE = "pendiente"
ESTADO_COMPLETADA = "completada"

# Mapas de prioridad para ordenar y mostrar.
PRIORIDADES = {"baja": 1, "media": 2, "alta": 3}
NOMBRE_PRIORIDAD = {1: "baja", 2: "media", 3: "alta"}

def parse_priority(value):
    """Convierte un texto de prioridad a su valor numérico."""
    if value is None:
        return None
    value = value.lower().strip()
    if value in PRIORIDADES:
        return PRIORIDADES[value]
    raise ValueError(f"Prioridad inválida: {value}. Opciones: baja, media, alta.")

def parse_date(value):
    """Valida que la fecha tenga formato ISO YYYY-MM-DD."""
    if value is None:
        return None
    try:
        datetime.datetime.strptime(value, "%Y-%m-%d")
        return value
    except ValueError as exc:
        raise ValueError(f"Fecha inválida: {value}. Use AAAA-MM-DD.") from exc

def is_overdue(due_date):
    """Indica si una fecha límite ya pasó y no se completó."""
    if not due_date:
        return False
    today = datetime.date.today().isoformat()
    return due_date < today

def _priority_color(priority):
    """Devuelve el nombre de la prioridad con color semántico."""
    name = NOMBRE_PRIORIDAD.get(priority, str(priority)).upper()
    if not _HAS_COLOR:
        return name
    colors = {"alta": Fore.RED, "media": Fore.YELLOW, "baja": Fore.GREEN}
    return f"{colors.get(name.lower(), '')}{name}{Style.RESET_ALL}"

def _due_color(due_date, status):
    """Devuelve la fecha límite, resaltada en rojo si está vencida."""
    if not due_date:
        return "sin fecha"

```

```

if _HAS_COLOR and status == ESTADO_PENDIENTE and is_overdue(due_date):
    return f"{Fore.RED}{due_date}{Style.RESET_ALL}"
return due_date

class TaskManager:
    """Gestiona la persistencia y operaciones de la lista de tareas."""

    def __init__(self, data_file=None):
        self.data_file = Path(data_file) if data_file else DEFAULT_DATA_FILE
        self.data_dir = self.data_file.parent
        self._ensure_storage()

    def _ensure_storage(self):
        """Crea el directorio y archivo JSON si no existen."""
        self.data_dir.mkdir(parents=True, exist_ok=True)
        if not self.data_file.exists():
            self.data_file.write_text("[]", encoding="utf-8")

    def load(self):
        """Carga las tareas desde el archivo JSON."""
        self._ensure_storage()
        try:
            return json.loads(self.data_file.read_text(encoding="utf-8"))
        except json.JSONDecodeError as exc:
            print(f"Error al leer el archivo de datos: {exc}", file=sys.stderr)
            return []

    def save(self, tasks):
        """Guarda las tareas en el archivo JSON."""
        self._ensure_storage()
        self.data_file.write_text(
            json.dumps(tasks, indent=2, ensure_ascii=False), encoding="utf-8"
        )

    @staticmethod
    def _next_id(tasks):
        """Calcula el siguiente identificador numérico."""
        return max((t["id"] for t in tasks), default=0) + 1

    def add(self, title, description=None, priority="media", due_date=None):
        """Agrega una nueva tarea y devuelve su representación."""
        tasks = self.load()
        task = {
            "id": self._next_id(tasks),
            "titulo": title,
            "descripcion": description or "",
            "prioridad": parse_priority(priority) or PRIORIDADES["media"],
            "fecha_limite": parse_date(due_date),
            "estado": ESTADO_PENDIENTE,
            "creada": datetime.datetime.now().isoformat(timespec="seconds"),
        }
        tasks.append(task)
        self.save(tasks)
        return task

    def list(self, status=None, priority=None, sort_by="creada"):
        """Devuelve las tareas filtradas y ordenadas."""
        tasks = self.load()

        if status and status != "todas":
            if status == "pendientes":
                tasks = [t for t in tasks if t["estado"] == ESTADO_PENDIENTE]
            elif status == "completadas":
                tasks = [t for t in tasks if t["estado"] == ESTADO_COMPLETADA]
            elif status == "hoy":
                today = datetime.date.today().isoformat()
                tasks = [t for t in tasks if t["fecha_limite"] == today]
            elif status == "vencidas":
                tasks = [
                    t

```



```

        for t in tasks:
            if t["estado"] == ESTADO_PENDIENTE and is_overdue(t["fecha_limite"])
        ]

    if priority:
        p = parse_priority(priority)
        tasks = [t for t in tasks if t["prioridad"] == p]

    if sort_by == "prioridad":
        tasks = sorted(tasks, key=lambda t: t["prioridad"], reverse=True)
    elif sort_by == "fecha":
        tasks = sorted(tasks, key=lambda t: t["fecha_limite"] or "")
    else:
        tasks = sorted(tasks, key=lambda t: t["id"])

    return tasks

def get(self, task_id):
    """Busca una tarea por ID."""
    for t in self.load():
        if t["id"] == task_id:
            return t
    return None

def complete(self, task_id):
    """Marca una tarea como completada. Devuelve True si existía."""
    tasks = self.load()
    for t in tasks:
        if t["id"] == task_id:
            t["estado"] = ESTADO_COMPLETADA
            t["completada"] = datetime.datetime.now().isoformat(timespec="seconds")
            self.save(tasks)
            return True
    return False

def delete(self, task_id):
    """Elimina una tarea por ID. Devuelve True si se eliminó."""
    tasks = self.load()
    new_tasks = [t for t in tasks if t["id"] != task_id]
    if len(new_tasks) == len(tasks):
        return False
    self.save(new_tasks)
    return True

def edit(self, task_id, title=None, description=None, priority=None, due_date=None):
    """Edita una tarea existente. Devuelve True si existía."""
    tasks = self.load()
    for t in tasks:
        if t["id"] == task_id:
            if title:
                t["titulo"] = title
            if description is not None:
                t["descripcion"] = description
            if priority:
                t["prioridad"] = parse_priority(priority)
            if due_date:
                t["fecha_limite"] = parse_date(due_date)
            t["actualizada"] = datetime.datetime.now().isoformat(timespec="seconds")
            self.save(tasks)
            return True
    return False

def search(self, query):
    """Busca tareas por palabra clave en título o descripción."""
    q = query.lower()
    return [
        t
        for t in self.load()
        if q in t["titulo"].lower() or q in t["descripcion"].lower()
    ]

```

```

def stats(self):
    """Calcula estadísticas del listado de tareas."""
    tasks = self.load()
    total = len(tasks)
    done = sum(1 for t in tasks if t["estado"] == ESTADO_COMPLETADA)
    pending = total - done
    overdue = sum(
        1
        for t in tasks
        if t["estado"] == ESTADO_PENDIENTE and is_overdue(t["fecha_limite"])
    )
    return {
        "total": total,
        "completadas": done,
        "pendientes": pending,
        "vencidas": overdue,
        "progreso": (done / total * 100) if total else 0.0,
    }

def export_csv(self, path):
    """Exporta las tareas a un archivo CSV. Devuelve la cantidad exportada."""
    tasks = self.load()
    if not tasks:
        return 0
    path = Path(path)
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(
            ["id", "titulo", "descripcion", "prioridad", "fecha_limite", "estado", "crea
da"]
        )
        for t in tasks:
            writer.writerow(
                [
                    t["id"],
                    t["titulo"],
                    t["descripcion"],
                    NOMBRE_PRIORIDAD.get(t["prioridad"], t["prioridad"]),
                    t["fecha_limite"] or "",
                    t["estado"],
                    t["creada"],
                ]
            )
    return len(tasks)

def _get_manager():
    """Devuelve una instancia del gestor con la ruta por defecto."""
    return TaskManager()

def _format_task(task):
    """Formatea una tarea para mostrarla en la terminal."""
    marker = "x" if task["estado"] == ESTADO_COMPLETADA else " "
    due = _due_color(task["fecha_limite"], task["estado"])
    priority = _priority_color(task["prioridad"])
    lines = [
        f"[{marker}] #{task['id']:>3} | {task['titulo']:<30} "
        f"| Prioridad: {priority:<8} | Fecha: {due}"
    ]
    if task["descripcion"]:
        lines.append(f"    {task['descripcion']}")
    return "\n".join(lines)

def _print_tasks(tasks):
    """Imprime una lista de tareas formateadas."""
    if not tasks:
        print("No hay tareas que coincidan con el filtro.")
        return
    for t in tasks:

```

```

        print(_format_task(t))

def _error(message):
    """Imprime un mensaje de error y termina con código de salida 1."""
    print(message, file=sys.stderr)
    sys.exit(1)

def cmd_add(args):
    """Comando: agregar tarea."""
    try:
        task = _get_manager().add(
            title=args.title,
            description=args.description,
            priority=args.priority,
            due_date=args.due,
        )
        print(f"Tarea agregada: #{task['id']} - {task['titulo']}")
    except ValueError as exc:
        _error(f"Error: {exc}")

def cmd_list(args):
    """Comando: listar tareas."""
    try:
        manager = _get_manager()
        if not manager.list():
            print("No hay tareas registradas.")
            return
        tasks = manager.list(
            status=args.status, priority=args.priority, sort_by=args.sort
        )
        _print_tasks(tasks)
    except ValueError as exc:
        _error(f"Error: {exc}")

def cmd_done(args):
    """Comando: completar tarea."""
    if _get_manager().complete(args.id):
        print(f"Tarea #{args.id} marcada como completada.")
    else:
        _error(f"No se encontró la tarea #{args.id}.")

def cmd_delete(args):
    """Comando: eliminar tarea."""
    if _get_manager().delete(args.id):
        print(f"Tarea #{args.id} eliminada.")
    else:
        _error(f"No se encontró la tarea #{args.id}.")

def cmd_edit(args):
    """Comando: editar tarea."""
    try:
        if _get_manager().edit(
            task_id=args.id,
            title=args.title,
            description=args.description,
            priority=args.priority,
            due_date=args.due,
        ):
            print(f"Tarea #{args.id} actualizada.")
        else:
            _error(f"No se encontró la tarea #{args.id}.")
    except ValueError as exc:
        _error(f"Error: {exc}")

```

```

def cmd_search(args):
    """Comando: buscar tareas."""
    tasks = _get_manager().search(args.query)
    if not tasks:
        print("No se encontraron coincidencias.")
        return
    for t in tasks:
        marker = "x" if t["estado"] == ESTADO_COMPLETADA else " "
        print(f"[{marker}] #{t['id']} | {t['título']} | {t['estado']}")

def cmd_stats(args):
    """Comando: estadísticas."""
    stats = _get_manager().stats()
    print(f"Total de tareas: {stats['total']}")
    print(f"Completadas: {stats['completadas']}")
    print(f"Pendientes: {stats['pendientes']}")
    print(f"Vencidas: {stats['vencidas']}")
    if stats["total"]:
        print(f"Progreso: {stats['progreso']:.1f}%")

def cmd_export(args):
    """Comando: exportar a CSV."""
    count = _get_manager().export_csv(args.path)
    if count:
        print(f"Exportadas {count} tareas a {args.path}")
    else:
        print("No hay tareas para exportar.")

def _build_parser():
    """Construye el parser de argumentos con todos los subcomandos."""
    parser = argparse.ArgumentParser(
        prog="todo",
        description="Lista de tareas multiplataforma desde la terminal.",
    )
    sub = parser.add_subparsers(dest="command", required=True)

    add = sub.add_parser("add", help="Agregar una tarea")
    add.add_argument("title", help="Título de la tarea")
    add.add_argument("-d", "--description", help="Descripción opcional")
    add.add_argument(
        "-p", "--priority", choices=["baja", "media", "alta"], default="media"
    )
    add.add_argument("--due", help="Fecha límite en formato AAAA-MM-DD")
    add.set_defaults(func=cmd_add)

    listar = sub.add_parser("list", help="Listar tareas")
    listar.add_argument(
        "--status",
        choices=["todas", "pendientes", "completadas", "hoy", "vencidas"],
        default="todas",
    )
    listar.add_argument("--priority", choices=["baja", "media", "alta"])
    listar.add_argument(
        "--sort", choices=["creada", "prioridad", "fecha"], default="creada"
    )
    listar.set_defaults(func=cmd_list)

    done = sub.add_parser("done", help="Marcar una tarea como completada")
    done.add_argument("id", type=int, help="ID de la tarea")
    done.set_defaults(func=cmd_done)

    delete = sub.add_parser("delete", help="Eliminar una tarea")
    delete.add_argument("id", type=int, help="ID de la tarea")
    delete.set_defaults(func=cmd_delete)

    edit = sub.add_parser("edit", help="Editar una tarea")
    edit.add_argument("id", type=int, help="ID de la tarea")
    edit.add_argument("-t", "--title", help="Nuevo título")

```

```

edit.add_argument("-d", "--description", help="Nueva descripción")
edit.add_argument("-p", "--priority", choices=["baja", "media", "alta"])
edit.add_argument("--due", help="Nueva fecha límite AAAA-MM-DD")
edit.set_defaults(func=cmd_edit)

search = sub.add_parser("search", help="Buscar tareas por palabra clave")
search.add_argument("query", help="Texto a buscar")
search.set_defaults(func=cmd_search)

stats = sub.add_parser("stats", help="Ver estadísticas")
stats.set_defaults(func=cmd_stats)

export = sub.add_parser("export", help="Exportar tareas a CSV")
export.add_argument("path", help="Ruta del archivo CSV de salida")
export.set_defaults(func=cmd_export)

return parser

def main():
    """Punto de entrada de la aplicación."""
    parser = _build_parser()
    args = parser.parse_args()
    args.func(args)

if __name__ == "__main__":
    main()

```

5.2. Pruebas unitarias - tests/test_todo.py

Se implementó una suite de 23 pruebas unitarias que cubren validaciones, CRUD, búsqueda, filtros, ordenamiento, detección de tareas vencidas, exportación CSV y persistencia.

```

"""Pruebas unitarias para la aplicación de lista de tareas."""

import sys
import tempfile
import unittest
from pathlib import Path

# Permite importar el módulo principal del proyecto.
sys.path.insert(0, str(Path(__file__).parent.parent))

from todo import (
    ESTADO_COMPLETADA,
    ESTADO_PENDIENTE,
    PRIORIDADES,
    TaskManager,
    is_overdue,
    parse_date,
    parse_priority,
)

class TestValidaciones(unittest.TestCase):
    """Valida las funciones de parseo de prioridad y fecha."""

    def test_parse_priority_validas(self):
        self.assertEqual(parse_priority("baja"), PRIORIDADES["baja"])
        self.assertEqual(parse_priority("MEDIA"), PRIORIDADES["media"])
        self.assertEqual(parse_priority(" alta "), PRIORIDADES["alta"])

    def test_parse_priority_invalida(self):
        with self.assertRaises(ValueError):
            parse_priority("urgente")

    def test_parse_priority_none(self):

```

```

        self.assertIsNone(parse_priority(None))

    def test_parse_date_valida(self):
        self.assertEqual(parse_date("2026-07-05"), "2026-07-05")

    def test_parse_date_invalida(self):
        with self.assertRaises(ValueError):
            parse_date("05-07-2026")

    def test_parse_date_none(self):
        self.assertIsNone(parse_date(None))

class TestFechasVencidas(unittest.TestCase):
    """Valida la detección de tareas vencidas."""

    def test_is_overdue_pasada(self):
        self.assertTrue(is_overdue("2020-01-01"))

    def test_is_overdue_hoy(self):
        from datetime import date

        self.assertFalse(is_overdue(date.today().isoformat()))

    def test_is_overdue_sin_fecha(self):
        self.assertFalse(is_overdue(None))
        self.assertFalse(is_overdue(""))

class TestTaskManager(unittest.TestCase):
    """Prueba el gestor de tareas con un archivo temporal."""

    def setUp(self):
        self.tempdir = tempfile.TemporaryDirectory()
        self.data_file = Path(self.tempdir.name) / "tasks.json"
        self.manager = TaskManager(str(self.data_file))

    def tearDown(self):
        self.tempdir.cleanup()

    def test_add_and_list(self):
        t1 = self.manager.add("Tarea 1", priority="alta", due_date="2026-07-10")
        t2 = self.manager.add("Tarea 2", description="desc", priority="baja")
        tasks = self.manager.list()
        self.assertEqual(len(tasks), 2)
        self.assertEqual(tasks[0]["id"], t1["id"])
        self.assertEqual(tasks[1]["titulo"], t2["titulo"])

    def test_complete(self):
        task = self.manager.add("Tarea")
        self.assertTrue(self.manager.complete(task["id"]))
        self.assertFalse(self.manager.complete(999))
        completed = self.manager.list(status="completadas")
        self.assertEqual(len(completed), 1)
        self.assertEqual(completed[0]["estado"], ESTADO_COMPLETADA)

    def test_delete(self):
        task = self.manager.add("Tarea")
        self.assertTrue(self.manager.delete(task["id"]))
        self.assertFalse(self.manager.delete(task["id"]))
        self.assertEqual(len(self.manager.list()), 0)

    def test_edit(self):
        task = self.manager.add("Tarea")
        self.assertTrue(
            self.manager.edit(
                task["id"], title="Nuevo título", priority="media", due_date="2026-08-01"
            )
        )
        updated = self.manager.get(task["id"])
        self.assertEqual(updated["titulo"], "Nuevo título")

```

```

        self.assertEqual(updated["prioridad"], PRIORIDADES["media"])
        self.assertEqual(updated["fecha_limite"], "2026-08-01")

    def test_edit_no_existente(self):
        self.assertFalse(self.manager.edit(999, title="Nuevo"))

    def test_search(self):
        self.manager.add("Comprar leche")
        self.manager.add("Llamar al banco", description="urgente")
        results = self.manager.search("leche")
        self.assertEqual(len(results), 1)
        self.assertEqual(results[0]["titulo"], "Comprar leche")
        results = self.manager.search("banco")
        self.assertEqual(len(results), 1)

    def test_search_case_insensitive(self):
        self.manager.add("Estudiar Python")
        results = self.manager.search("PYTHON")
        self.assertEqual(len(results), 1)

    def test_stats(self):
        self.manager.add("T1")
        t2 = self.manager.add("T2")
        self.manager.complete(t2["id"])
        stats = self.manager.stats()
        self.assertEqual(stats["total"], 2)
        self.assertEqual(stats["completadas"], 1)
        self.assertEqual(stats["pendientes"], 1)
        self.assertEqual(stats["vencidas"], 0)
        self.assertEqual(stats["progreso"], 50.0)

    def test_stats_vencidas(self):
        self.manager.add("T1", due_date="2020-01-01")
        stats = self.manager.stats()
        self.assertEqual(stats["vencidas"], 1)

    def test_list_filters(self):
        t1 = self.manager.add("T1", priority="alta")
        self.manager.add("T2", priority="baja")
        self.manager.complete(t1["id"])
        self.assertEqual(len(self.manager.list(status="completadas")), 1)
        self.assertEqual(len(self.manager.list(status="pendientes")), 1)
        self.assertEqual(len(self.manager.list(priority="alta")), 1)
        self.assertEqual(len(self.manager.list(priority="baja")), 1)

    def test_list_sort(self):
        self.manager.add("A", priority="baja", due_date="2026-08-01")
        self.manager.add("B", priority="alta", due_date="2026-07-01")
        by_priority = self.manager.list(sort_by="prioridad")
        self.assertEqual(by_priority[0]["prioridad"], PRIORIDADES["alta"])
        by_date = self.manager.list(sort_by="fecha")
        self.assertEqual(by_date[0]["fecha_limite"], "2026-07-01")

    def test_export_csv(self):
        self.manager.add("T1")
        self.manager.add("T2", description="desc")
        export_path = Path(self.tempdir.name) / "out.csv"
        count = self.manager.export_csv(str(export_path))
        self.assertEqual(count, 2)
        self.assertTrue(export_path.exists())
        content = export_path.read_text(encoding="utf-8")
        self.assertIn("id,titulo,descripcion,prioridad,fecha_limite,estado,creada", content)
        self.assertIn("T2", content)

    def test_export_csv_sin_tareas(self):
        export_path = Path(self.tempdir.name) / "empty.csv"
        count = self.manager.export_csv(str(export_path))
        self.assertEqual(count, 0)
        self.assertFalse(export_path.exists())

    def test_data_persistence(self):

```

```

        task = self.manager.add("Persistente")
        manager2 = TaskManager(str(self.data_file))
        self.assertIsNotNone(manager2.get(task["id"]))

if __name__ == "__main__":
    unittest.main()

```

6. Evidencia de ejecución

A continuación se incluyen las capturas de las pruebas manuales realizadas en la terminal. Los bloques simulan el fondo negro y la tipografía de consola.

Capturas de verificación - Taller 1

A continuación se documentan las pruebas realizadas en la terminal para cada uno de los prompts. Se aplican criterios de calidad (QA) y experiencia de usuario (UX). Los datos se almacenan en %USERPROFILE%\todo_cli\tasks.json.

Prompt 1 - Arquitectura, dominio, persistencia y pruebas

Ejecución de la suite de pruebas unitarias

```

$ python -m unittest discover tests
.....
-----
Ran 23 tests in 0.090s

OK

```

Persistencia y recuperación de JSON corrupto

Si el archivo JSON se corrompe, la aplicación se recupera devolviendo una lista vacía.

Prompt 2 - Interfaz de línea de comandos (CLI)

Ayuda general

```

$ python todo.py --help
usage: todo [-h] {add,list,done,delete,edit,search,stats,export} ...

Lista de tareas multiplataforma desde la terminal.

positional arguments:
  {add,list,done,delete,edit,search,stats,export}
    add                Agregar una tarea
    list               Listar tareas
    done               Marcar una tarea como completada
    delete            Eliminar una tarea
    edit               Editar una tarea
    search             Buscar tareas por palabra clave
    stats              Ver estadísticas
    export             Exportar tareas a CSV

options:
  -h, --help          show this help message and exit

```


Manejo de errores

```
$ python todo.py add "Test" -p invalida
usage: todo add [-h] [-d DESCRIPTION] [-p {baja,media,alta}] [--due DUE] title
todo add: error: argument -p/--priority: invalid choice: 'invalida' (choose from baja, media, alta)
Exit: 2

$ python todo.py done 999
No se encontró la tarea #999.
Exit: 1
```

Prompt 3 - Funcionalidades avanzadas, filtros y control de calidad

Agregar tareas con prioridad y fecha límite

```
$ python todo.py add "Estudiar prompts" -d "Repasar estructura" -p alta --due 2026-07-02
Tarea agregada: #1 - Estudiar prompts

$ python todo.py add "Hacer ejercicio" -p media
Tarea agregada: #2 - Hacer ejercicio

$ python todo.py add "Pagar servicios" -p alta --due 2026-07-01
Tarea agregada: #3 - Pagar servicios

$ python todo.py add "Leer libro" -p baja --due 2026-07-10
Tarea agregada: #4 - Leer libro
```

Listado general

```
$ python todo.py list
[ ] # 1 | Estudiar prompts | Prioridad: ALTA | Fecha: 2026-07-02
    Repasar estructura
[ ] # 2 | Hacer ejercicio | Prioridad: MEDIA | Fecha: sin fecha
[ ] # 3 | Pagar servicios | Prioridad: ALTA | Fecha: 2026-07-01
[ ] # 4 | Leer libro | Prioridad: BAJA | Fecha: 2026-07-10
```

Filtros por estado

```
$ python todo.py list --status hoy
[ ] # 1 | Estudiar prompts | Prioridad: ALTA | Fecha: 2026-07-02
    Repasar estructura

$ python todo.py list --status vencidas
[ ] # 3 | Pagar servicios | Prioridad: ALTA | Fecha: 2026-07-01

$ python todo.py list --status pendientes
[ ] # 2 | Hacer ejercicio | Prioridad: MEDIA | Fecha: sin fecha
[ ] # 3 | Pagar servicios | Prioridad: ALTA | Fecha: 2026-07-01
[ ] # 4 | Leer libro | Prioridad: BAJA | Fecha: 2026-07-10
```

Filtros por prioridad y ordenamiento

```
$ python todo.py list --priority alta
[ ] # 1 | Estudiar prompts | Prioridad: ALTA | Fecha: 2026-07-02
    Repasar estructura
[ ] # 3 | Pagar servicios | Prioridad: ALTA | Fecha: 2026-07-01

$ python todo.py list --sort fecha
[ ] # 3 | Pagar servicios | Prioridad: ALTA | Fecha: 2026-07-01
[ ] # 1 | Estudiar prompts | Prioridad: ALTA | Fecha: 2026-07-02
    Repasar estructura
[ ] # 4 | Leer libro | Prioridad: BAJA | Fecha: 2026-07-10
[ ] # 2 | Hacer ejercicio | Prioridad: MEDIA | Fecha: sin fecha
```

Búsqueda y edición

```
$ python todo.py search "libro"
[ ] #4 | Leer libro | pendiente

$ python todo.py edit 2 --due 2026-07-05
Tarea #2 actualizada.
```

Prompt 4 - UX profesional, colores, exportación CSV y documentación

Estadísticas

```
$ python todo.py done 1
Tarea #1 marcada como completada.

$ python todo.py stats
Total de tareas: 4
Completadas: 1
Pendientes: 3
Vencidas: 1
Progreso: 25.0%
```

Exportación a CSV

```
$ python todo.py export tareas.csv
Exportadas 4 tareas a tareas.csv

$ Get-Content tareas.csv -Encoding UTF8
id,titulo,descripcion,prioridad,fecha_limite,estado,creada
1,Estudiar prompts,Repasar estructura,alta,2026-07-02,completada,2026-07-02T22:56:51
2,Hacer ejercicio,,media,2026-07-05,pendiente,2026-07-02T22:56:51
3,Pagar servicios,,alta,2026-07-01,pendiente,2026-07-02T22:56:51
4,Leer libro,,baja,2026-07-10,pendiente,2026-07-02T22:56:51
```

Colores en terminal

Si se instala colorama (pip install -r requirements.txt), las prioridades se muestran con colores semánticos:

- **Alta:** rojo
- **Media:** amarillo
- **Baja:** verde
- **Fechas vencidas:** rojo (solo para tareas pendientes)

Sin colorama, el programa funciona exactamente igual en texto plano.

7. Documentación del proyecto

7.1. README.md

Lista de Tareas - Terminal (Python)

Aplicación profesional de lista de tareas (Todo List) que se ejecuta completamente desde la terminal. Construida con prompt engineering, enfoque de calidad (QA) y buena experiencia de usuario (UX).

Requisitos

- Python 3.10 o superior (probado en Python 3.14)
- (Opcional) colorama para colores en Windows: `pip install -r requirements.txt`

Instalación

```
pip install -r requirements.txt
```

Uso básico

```
python todo.py add "Estudiar prompts" -p alta --due 2026-07-05
python todo.py add "Hacer ejercicio" -d "30 minutos" -p media
python todo.py list
python todo.py done 1
python todo.py stats
python todo.py export tareas.csv
```

Comandos disponibles

Comando	Descripción
add	Agregar una tarea
list	Listar tareas con filtros y ordenamiento
done	Marcar tarea como completada
delete	Eliminar una tarea
edit	Editar una tarea
search	Buscar tareas por palabra clave
stats	Ver estadísticas
export	Exportar tareas a CSV

Filtros y ordenamiento

```
# Filtrar por estado
python todo.py list --status pendientes
python todo.py list --status completadas
python todo.py list --status hoy
python todo.py list --status vencidas

# Filtrar por prioridad
python todo.py list --priority alta

# Ordenar resultados
python todo.py list --sort prioridad
python todo.py list --sort fecha
```

Persistencia

Las tareas se guardan en un archivo JSON en el directorio personal del usuario:

```
%USERPROFILE%\todo_cli\tasks.json
```

Calidad y pruebas

El proyecto incluye una suite de pruebas unitarias en `tests/test_todo.py`.

```
python -m unittest discover tests
```

Las pruebas cubren validaciones, CRUD, filtros, búsqueda, ordenamiento, detección de tareas vencidas y exportación CSV.

Colores en terminal

Si instalas colorama, las prioridades se muestran con colores semánticos:

- **Alta:** rojo
- **Media:** amarillo
- **Baja:** verde
- **Fechas vencidas:** rojo (solo tareas pendientes)

Sin colorama, el programa funciona exactamente igual en texto plano.

Créditos

Generado mediante prompts estructurados con enfoque de QA y UX usando el asistente Cascade con OpenAI GPT-4o.

7.2. requirements.txt

```
colorama>=0.4.6
```

7.3. .gitignore

```
__pycache__/  
*.py[cod]  
*.egg-info/  
.env  
.venv/  
venv/  
*.csv
```

8. Plan de calidad y conclusiones

1. **Arquitectura:** La lógica de negocio está separada de la interfaz CLI mediante la clase TaskManager, facilitando las pruebas y el mantenimiento.
2. **Persistencia:** Las tareas se almacenan en un archivo JSON en el directorio personal del usuario (%USERPROFILE%\todo_cli\tasks.json). El sistema se recupera automáticamente ante archivos corruptos.
3. **Validación:** Se validan prioridades (baja, media, alta) y fechas en formato ISO (AAAA-MM-DD). Los errores se reportan por stderr con códigos de salida distintos de cero.
4. **Pruebas:** 23 pruebas unitarias se ejecutan exitosamente con `python -m unittest discover tests`.
5. **UX:** Colores semánticos opcionales, tabla alineada, fechas vencidas resaltadas y mensajes de error informativos.
6. **Entrega:** El proyecto se encuentra en GitHub y es ejecutable en Python 3.14 desde la terminal.